PAC-MAN GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive Pac-Man game using Python Tkinter that allows players to navigate a maze, collect dots and power pellets, and avoid ghosts. The game includes keyboard controls, automatic score tracking, and lives management. It features collision detection, win and game-over messages, and restart functionality. The program demonstrates GUI design, event-driven programming, animation, and basic game development concepts.

PROCEDURE

1. Start the program and import the required Python libraries: tkinter and random.
2. Create the main game window using Tkinter and set the title as "PAC-MAN".
3. Design a graphical maze using the Canvas widget with coloured walls and a dark background.
4. Create the Pac-Man character, ghosts, dots, and power pellets within the maze.
5. Implement keyboard controls using arrow keys or WASD to move Pac-Man in four directions.
6. Update character and ghost movements continuously using Tkinter's after() method.
7. Detect dot and power pellet collection and update the player's score.
8. Implement collision detection between Pac-Man and ghosts, including life reduction and power pellet effects.
9. Display the Game Over message when all lives are lost and a You Win message when all dots are collected.
10. Provide restart, pause/resume, and exit controls for managing the game.
11. Display the player's score, remaining lives, and dots left on the game interface.
12. Execute the game and verify movement, scoring, ghost behaviour, collision detection, and game controls.

PROGRAM CODE

```
import tkinter as tk
import random

# Pac-Man-inspired maze game using Python's built-in Tkinter.
TILE = 24
COLS = 25
ROWS = 21
```

```
WIDTH = COLS * TILE
HEIGHT = ROWS * TILE
TICK_MS = 110

MAZE_TEMPLATE = [
    "############################",
    "#...........#...........#",
    "#.###.######.#.######.###.#",
    "#o###.######.#.######.###o#",
    "#..........................#",
    "#.###.#.##########.#.###..#",
    "#.....#....#....#.#......#",
    "######.#####.#.#####.######..#",
    "#.........#.............#",
    "#.###.###.#.###.###.###..#",
    "#...#.....G.....#...#....#",
    "###.#.###.#.###.#.###.####",
    "#.....#...#...#...#.......#",
    "#.###.#.######.#.###.###...#",
    "#o..#.................#..o#",
    "###.#.######.#.######.#.######",
    "#.....#.....#.....#........#",
    "#.##########.#.############..#",
    "#..........................#",
    "#...........#............#",
    "############################",
]


class PacManGame:
    def __init__(self, root):
        self.root = root
        self.root.title("PAC-MAN")
        self.root.resizable(False, False)
        self.root.configure(bg="#111827")

        tk.Label(
            root, text="PAC-MAN", font=("Arial Black", 22),
            bg="#111827", fg="#FACC15"
        ).pack(pady=(10, 2))

        self.hud = tk.Label(
            root, text="SCORE: 0    |    LIVES: 3",
            font=("Consolas", 12, "bold"), bg="#111827", fg="white"
```

```
        )
        self.hud.pack(pady=(0, 8))

        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT, bg="#050816",
            highlightthickness=2, highlightbackground="#38BDF8"
        )
        self.canvas.pack(padx=12)

        buttons = tk.Frame(root, bg="#111827")
        buttons.pack(pady=8)
        tk.Button(
            buttons, text="RESTART", command=self.restart,
            font=("Arial", 10, "bold"), bg="#22C55E", fg="#052E16"
        ).pack(side="left", padx=5)
        tk.Button(
            buttons, text="PAUSE / RESUME", command=self.toggle_pause,
            font=("Arial", 10, "bold"), bg="#38BDF8", fg="#082F49"
        ).pack(side="left", padx=5)
        tk.Button(
            buttons, text="EXIT", command=root.destroy,
            font=("Arial", 10, "bold"), bg="#EF4444", fg="white"
        ).pack(side="left", padx=5)

        tk.Label(
            root, text="Move: Arrow keys / WASD   |   Eat all dots and avoid ghosts!",
            font=("Arial", 9), bg="#111827", fg="#CBD5E1"
        ).pack(pady=(0, 10))

        self.root.bind("<KeyPress>", self.key_press)
        self.timer = None
        self.restart()

    def restart(self):
        if self.timer is not None:
            try:
                self.root.after_cancel(self.timer)
            except tk.TclError:
                pass
        self.timer = None
        self.maze = [list(row.ljust(COLS, "#")[:COLS]) for row in
MAZE_TEMPLATE]
        self.player = [1, 1]
        self.direction = (0, 0)
```

```
    self.next_direction = (0, 0)
    self.score = 0
    self.lives = 3
    self.paused = False
    self.running = True
    self.won = False
    self.tick_count = 0
    self.dots_left = sum(cell in (".", "o") for row in self.maze for cell in row)
    self.ghosts = [
        {"x": 12, "y": 10, "color": "#F87171"},
        {"x": 13, "y": 10, "color": "#60A5FA"},
        {"x": 11, "y": 10, "color": "#F0ABFC"},
    ]
    # Clear ghost starting cells from maze.
    for ghost in self.ghosts:
        if self.maze[ghost["y"]][ghost["x"]] != "#":
            self.maze[ghost["y"]][ghost["x"]] = " "
    self.update_hud()
    self.draw()
    self.loop()

def key_press(self, event):
    key = event.keysym.lower()
    if key == "p":
        self.toggle_pause()
        return
    if key == "r" and not self.running:
        self.restart()
        return
    if not self.running or self.paused:
        return
    directions = {
        "up": (0, -1), "w": (0, -1),
        "down": (0, 1), "s": (0, 1),
        "left": (-1, 0), "a": (-1, 0),
        "right": (1, 0), "d": (1, 0)
    }
    if key in directions:
        self.next_direction = directions[key]

def is_wall(self, x, y):
    if x < 0 or x >= COLS or y < 0 or y >= ROWS:
        return True
    return self.maze[y][x] == "#"
```

```
def move_player(self):
    if self.next_direction != (0, 0):
        nx = self.player[0] + self.next_direction[0]
        ny = self.player[1] + self.next_direction[1]
        if not self.is_wall(nx, ny):
            self.direction = self.next_direction

    dx, dy = self.direction
    nx, ny = self.player[0] + dx, self.player[1] + dy
    if not self.is_wall(nx, ny):
        self.player = [nx, ny]

    x, y = self.player
    cell = self.maze[y][x]
    if cell == ".":
        self.maze[y][x] = " "
        self.score += 10
        self.dots_left -= 1
    elif cell == "o":
        self.maze[y][x] = " "
        self.score += 50
        self.dots_left -= 1
        # Power pellet: briefly slows ghosts by skipping their next move.
        self.power_ticks = 18

    if self.dots_left <= 0:
        self.running = False
        self.won = True

def move_ghosts(self):
    if getattr(self, "power_ticks", 0) > 0:
        self.power_ticks -= 1
        if self.power_ticks % 2 == 0:
            return

    for ghost in self.ghosts:
        options = [(0, -1), (0, 1), (-1, 0), (1, 0)]
        random.shuffle(options)
        gx, gy = ghost["x"], ghost["y"]
        valid = [(gx + dx, gy + dy) for dx, dy in options
                 if not self.is_wall(gx + dx, gy + dy)]
        if valid:
            # Often chase the player, but occasionally choose randomly.
```

```
            if random.random() < 0.65:
                valid.sort(key=lambda pos: abs(pos[0] - self.player[0]) + abs(pos[1] -
self.player[1]))
                choice = valid[0]
            else:
                choice = random.choice(valid)
            ghost["x"], ghost["y"] = choice

    def check_collision(self):
        px, py = self.player
        for ghost in self.ghosts:
            if ghost["x"] == px and ghost["y"] == py:
                if getattr(self, "power_ticks", 0) > 0:
                    ghost["x"], ghost["y"] = 12, 10
                    self.score += 200
                else:
                    self.lives -= 1
                    if self.lives <= 0:
                        self.running = False
                    else:
                        self.player = [1, 1]
                        self.direction = (0, 0)
                        self.next_direction = (0, 0)
                    break

    def loop(self):
        if not self.running or self.paused:
            self.draw()
            return

        self.tick_count += 1
        self.move_player()
        if self.tick_count % 2 == 0:
            self.move_ghosts()
        self.check_collision()
        self.update_hud()
        self.draw()

        if self.running:
            self.timer = self.root.after(TICK_MS, self.loop)

    def update_hud(self):
        self.hud.config(text=f"SCORE: {self.score}  |  LIVES: {self.lives}  |  DOTS
LEFT: {self.dots_left}")
```

```
    def draw(self):
        self.canvas.delete("all")
        for y, row in enumerate(self.maze):
            for x, cell in enumerate(row):
                x1, y1 = x * TILE, y * TILE
                x2, y2 = x1 + TILE, y1 + TILE
                if cell == "#":
                    self.canvas.create_rectangle(
                        x1, y1, x2, y2, fill="#1729A8",
                        outline="#3B82F6", width=1
                    )
                else:
                    self.canvas.create_rectangle(x1, y1, x2, y2, fill="#050816", outline="")
                    if cell == ".":
                        self.canvas.create_oval(
                            x1 + 10, y1 + 10, x1 + 14, y1 + 14,
                            fill="#FDE68A", outline=""
                        )
                    elif cell == "o":
                        self.canvas.create_oval(
                            x1 + 5, y1 + 5, x1 + 19, y1 + 19,
                            fill="#FDE68A", outline="#FACC15", width=2
                        )

        # Draw Pac-Man as a yellow circle with a wedge mouth.
        px, py = self.player
        cx, cy = px * TILE + TILE / 2, py * TILE + TILE / 2
        dx, dy = self.direction
        if dx == 1:
            start, extent = 35, 290
        elif dx == -1:
            start, extent = 215, 290
        elif dy == -1:
            start, extent = 125, 290
        elif dy == 1:
            start, extent = 305, 290
        else:
            start, extent = 35, 290
        self.canvas.create_arc(
            cx - 10, cy - 10, cx + 10, cy + 10,
            start=start, extent=extent, fill="#FACC15", outline="#FACC15",
style=tk.PIESLICE
        )
```

```
    # Draw ghosts.
    for ghost in self.ghosts:
        gx, gy = ghost["x"] * TILE + TILE / 2, ghost["y"] * TILE + TILE / 2
        self.canvas.create_oval(
            gx - 10, gy - 10, gx + 10, gy + 10,
            fill=ghost["color"], outline="#F8FAFC", width=1
        )
        self.canvas.create_rectangle(
            gx - 10, gy, gx + 10, gy + 9,
            fill=ghost["color"], outline=""
        )
        for ex in (-4, 4):
            self.canvas.create_oval(gx + ex - 3, gy - 4, gx + ex + 2, gy + 2,
                                    fill="white", outline="")
            self.canvas.create_oval(gx + ex - 1, gy - 3, gx + ex + 1, gy + 1,
                                    fill="#111827", outline="")

    if self.paused:
        self.draw_overlay("PAUSED", "Press P or PAUSE / RESUME")
    elif self.won:
        self.draw_overlay("YOU WIN!", f"Final score: {self.score}")
    elif not self.running:
        self.draw_overlay("GAME OVER", f"Final score: {self.score}")

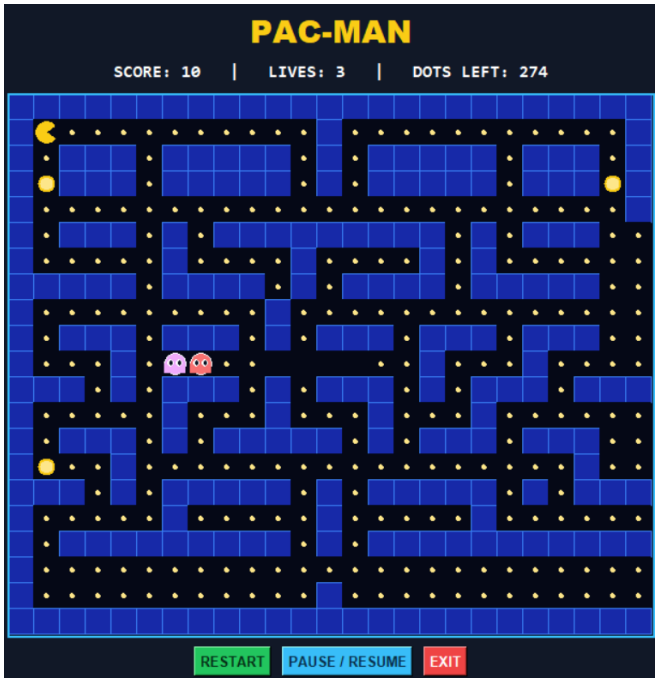
def draw_overlay(self, title, subtitle):
    self.canvas.create_rectangle(
        95, HEIGHT // 2 - 48, WIDTH - 95, HEIGHT // 2 + 48,
        fill="#111827", outline="#FACC15", width=3
    )
    self.canvas.create_text(
        WIDTH // 2, HEIGHT // 2 - 12, text=title,
        font=("Arial Black", 20), fill="#FACC15"
    )
    self.canvas.create_text(
        WIDTH // 2, HEIGHT // 2 + 18, text=subtitle,
        font=("Arial", 11, "bold"), fill="white"
    )

def toggle_pause(self):
    if not self.running:
        return
    self.paused = not self.paused
    if self.paused and self.timer is not None:
```

```
            try:
                self.root.after_cancel(self.timer)
            except tk.TclError:
                pass
            self.timer = None
        elif not self.paused:
            self.loop()
        self.draw()


if __name__ == "__main__":
    root = tk.Tk()
    game = PacManGame(root)
    root.mainloop()
```

output:





RESULT:

Thus, an interactive Pac-Man game was developed using Python Tkinter with graphical maze design and keyboard-based movement. The game includes random ghost movement, dot collection, automatic score calculation, and collision detection. It features lives tracking, win and game-over messages, restart functionality, pause/resume options, and exit controls. The program demonstrates GUI design, event-driven programming, animation, object-oriented programming, and basic game development concepts.